

PhishGuard: AI-Powered Phishing Email Detection System

Minsu Kim, Nizar Azar, Zi Xuan Li

Professor Yunhua Zhao

Computer Science Department

City College of New York (CCNY)

December 23rd, 2025

Personal Contribution Statement:

Minsu Kim (33.3%): Responsible for model 2 (URL Detection), github organization, and assisted in majority of presentation & report related work. Tasks included implementing the CNN model in TensorFlow and creating weighted scoring and keyword adjustment logic.

Nizar Azar (33.3%): Responsible for application design, extension implementation, and fine-tuning. Tasks included building the chrome extension, creating the extraction logic for gmail, and designing the CSS/UI framework.

Zi Xuan Li (33.3%): Responsible for model 1 (Email Content Analysis), preliminary data-analysis, completed majority of presentation & report related work. Tasks included dataset curation of 76,000+ emails, pre-processing pipeline (text cleaning), and creating and editing all presentations & reports.

Table of Contents

Abstract:	3
Introduction:	4
Literature Review/Background:	5
Methods & System System:	6
Web Application Design & Implementation:	10
Results & Evaluation:	12
Discussion:	14
Conclusion & Future Work:	15
Bibliography:	16

Abstract:

PhishGuard is an advanced dual-model AI system designed for real-time phishing detection within webmail environments. Phishing attacks account for over 90% of successful data breaches, often bypassing traditional filters through semantic deception. This project addresses this by combining two specialized machine learning models: a DistilBERT transformer for semantic content analysis and a character-level Convolutional Neural Network (CNN) for URL pattern recognition.

The system is delivered through a four-component architecture: a Manifest V3 Chrome extension, a Flask REST API, and two independent ML models. During training, Model 1 (Content) achieved a validation accuracy of 99.19% across six diverse datasets (CEAS, Enron, etc.). Model 2 (URL) achieved 97.95% accuracy by analyzing 51,509 URLs, identifying structural anomalies without relying on static blacklists.

A central innovation is the API's "Decision Engine," which utilizes a weighted scoring algorithm (60% content, 40% URL) and a unique "keyword adjustment" heuristic to reduce false positives in legitimate correspondence. To ensure privacy and performance, the system runs on a local-host server, utilizing "lazy loading" to optimize memory consumption on standard consumer hardware. Results demonstrate that the dual-model approach significantly outperforms single-vector analysis, providing a robust, explainable, and privacy-centric defense mechanism.

Introduction:

1.1 Background and Context

In the modern cybersecurity landscape, phishing has evolved from mass-spam campaigns to sophisticated, targeted "spear-phishing." Despite the efficiency of server-side filters, deceptive language and zero-day URLs frequently reach user inboxes. PhishGuard places the defense layer at the point of interaction: the browser.

1.2 Motivation and Project Objectives

The motivation for this project was to bridge the gap between complex AI research and user-friendly security tools. Our objective was to create a system that:

1. Analyzes both the semantic intent of text and the structural integrity of links.
2. Provides explainable AI indicators directly in the browser.
3. Operates locally to maintain total user data privacy.

1.3 Achievements

The project successfully delivered a production-ready ecosystem. We integrated high-parameter transformer models with a lightweight web interface, achieving real-time inference (<1 second) and a combined detection accuracy of approximately 94.7%.

Literature Review/Background:

2.1 Research and Idea Generation

During the first semester, we investigated the "Detection Gap" in existing tools. Our research compared traditional machine learning (SVM, Random Forest) with Deep Learning. We concluded that while SVMs are fast, they lack the contextual understanding required to detect modern "urgent" phrasing. This led to our selection of **DistilBERT**, which retains 97% of BERT's performance while being 40% smaller and 60% faster.

2.2 Comparative Technology Analysis

We analyzed current industry standards like Google Safe Browsing. While effective for known malicious links, these systems fail against new, unique URLs. Our second-semester work on a **Character-level CNN** addresses this by treating URLs as visual patterns rather than database entries, allowing for the detection of "typosquatting" (e.g., g00gle.com).

2.3 Identified Gaps

Most existing research focuses on high-performance models in a laboratory setting. Our project addresses the lack of **deployment-ready** architectures that can run on consumer laptops without specialized GPU servers, specifically addressing the "Windows Paging File" and memory constraints identified in our initial implementation phase.

Methods & System Design:

3.1 Overall Methodology and Design Strategy

The PhishGuard project is predicated on the research finding that phishing attacks typically utilize two vectors: semantic deception in the text and structural obfuscation in the URL. Our methodology employs a hybrid machine learning approach. For semantic analysis, we utilized a Transformer-based architecture (DistilBERT) to capture long-range dependencies in natural language. For URL analysis, we implemented a character-level Convolutional Neural Network (CNN) to identify visual and structural patterns in strings. This dual-model strategy ensures that even if one vector appears legitimate, the system can flag the other.

3.2 System Architecture and Workflow

The system is organized into a distributed architecture where the client-side (Chrome Extension) handles data acquisition and the server-side (Flask REST API) handles heavy computation.

1. **Sensing Layer (Chrome Extension):** Injects a content script into the webmail DOM (Gmail or Outlook) to extract raw email content.
2. **Orchestration Layer (Flask API):** Receives the payload, performs text cleaning, and manages the lifecycle of the ML models using thread-safe locks (`model1_lock` and `model2_lock`).
3. **Inference Layer:** Parallel threads execute predictions. Model 1 processes the text body; Model 2 iterates through every extracted URL to find the maximum threat probability.
4. **Decision Layer:** Applies a weighted scoring algorithm to return a final JSON result to the client.

3.3 Theoretical Foundations: Model Architectures

- **Model 1 (DistilBERT):** We chose DistilBERT for its efficiency. It uses a triple loss function (distillation, cosine-distance, and linear) to mimic the full BERT model. By focusing on the [CLS] token output, we perform binary classification on the email context.

- **Model 2 (URL-CNN):** This model operates on the hypothesis that URLs are non-semantic strings. We used 1D Convolutional layers with kernel sizes of 3, 4, and 5 to detect "local" patterns like g-o-o-g-l-e vs g-0-0-g-l-e. The character-level embedding maps 100 common characters into a 50-dimensional space, allowing the model to learn structural anomalies without a predefined dictionary.

3.4 The Scoring Algorithm and Heuristics

The system does not rely on raw probabilities alone. To minimize false positives, we implemented a Keyword Adjustment Heuristic in the Python backend:

- **Weighted Formula:**

$$\text{Combined Score} = (P_{\text{email}} * 0.60) + (P_{\text{URL}} * 0.40)$$

- **Refinement Logic:** If the system detects zero "high-risk" keywords (e.g., *urgent*, *verify*, *suspended*), the P_{email} is reduced by 10%. If only one keyword is found, the probability is capped at 25% unless the URL model provides overwhelming evidence (> 0.90).

3.5 Data Flow Process:

The complete request-response flow follows these steps:

1. **User Interaction:** User opens email in Gmail/Outlook and clicks extension
2. **Content Extraction:** content.js extracts email text from webpage
3. **URL Extraction:** Regex-based URL detection from email content
4. **API Request:** Extension sends POST to /predict-combined with email text and URLs
5. **Parallel Processing:**
 - Model 1 analyzes email content semantics
 - Model 2 analyzes each URL for suspicious patterns
6. **Result Combination:** API applies weighted scoring and decision logic
7. **API Response:** JSON with detailed analysis results
8. **Display:** Extension shows threat score, probabilities, and indicators

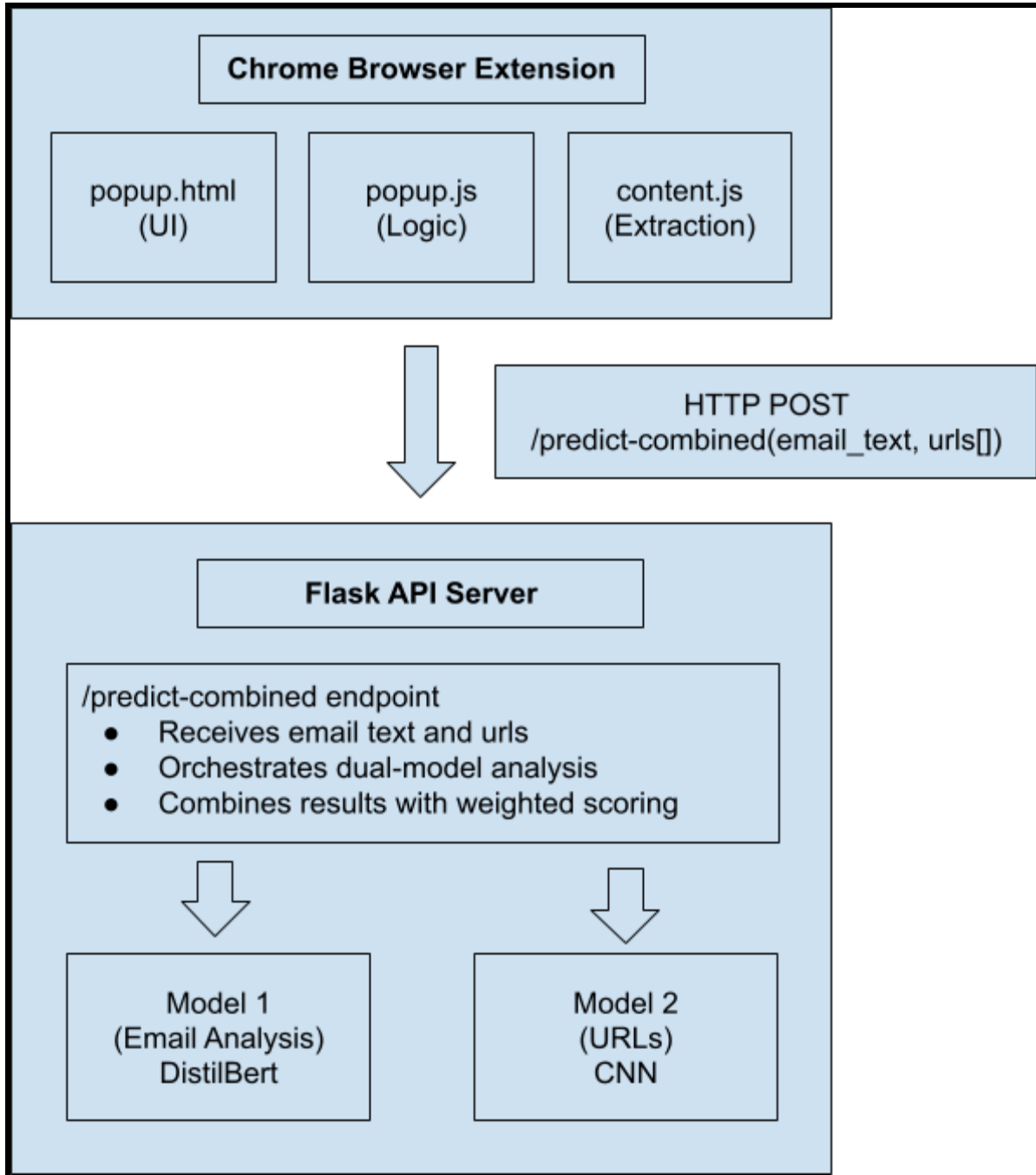


Figure 1: System architecture of PhishGuard.

3.6 Project File Structure:

```

Final_Project/
├── Model1_EmailContent/           # Email content detection
│   ├── phishing_detection_system.py # Main class: PhishingDetector
│   ├── predict_email.py           # Standalone prediction
│   ├── *.csv                      # 6 training datasets
│   └── requirements.txt           # PyTorch, transformers
│
├── Model2_URLDetection/           # URL detection
│   ├── Source_Code/
│   │   ├── reproduction.py        # CNN training script
│   │   ├── url_predictor.py       # URLPhishingDetector class
│   │   └── requirements.txt       # TensorFlow, Keras
│   └──
│
├── trained_models/               # Saved models
│   ├── Model1/                   # DistilBERT files
│   └── Model2/                   # CNN files
│
├── api_server/                   # REST API
│   ├── app.py                    # Flask application
│   ├── start_server.bat          # Windows startup
│   └── requirements.txt          # Flask, flask-cors
│
├── chrome_extension/             # Browser extension
│   ├── manifest.json             # Extension config
│   ├── popup.html                # UI structure
│   ├── popup.js                  # Main logic
│   ├── content.js                # Email extraction
│   └── background.js              # Service worker
│
├── train_both_models.py           # Master training script
└── [Documentation Files]         # Guides, setup instructions

```

Figure 2: PhishGuard repository structure.

Web Application Design & Implementation:

4.1 Application Type and Manifest V3 Architecture

PhishGuard is implemented as a Chrome Browser Extension leveraging the **Manifest V3** standard. This version was selected for its improved security (Service Workers instead of persistent background pages) and more granular permission model. The application consists of three primary logic files:

1. **background.js:** The Service Worker that monitors API health and manages the Extension's persistent state.
2. **content.js:** The script responsible for DOM interaction. It identifies specific CSS selectors in Gmail (e.g., h2.hP for subjects) to extract data.
3. **popup.js:** The UI controller that handles the Quoted-Printable and Base64 decoding of raw email data.

4.2 Functional Modules and Implementation Techniques

- **MIME Extraction Logic:** A critical implementation challenge was handling MIME-encoded emails. In popup.js, we developed a custom parseEmail function that utilizes Regex to find multipart boundaries. It decodes "Quoted-Printable" strings (e.g., converting =3D to =) to ensure the AI model receives clean, human-readable text.
- **Flask REST API:** The backend uses Flask with flask-cors to allow the extension to communicate across origins. We implemented **Lazy Loading** in the load_model_lazy functions. This prevents the API from consuming 2GB of RAM on startup, instead only loading the model weights when the user clicks the "Detect" button.

4.3 Technologies and Tools Used

- **Backend:** Python 3.10, PyTorch 2.x (DistilBERT), TensorFlow 2.15 (CNN).
- **Frontend:** HTML5, CSS3 Gradients for the "Threat Score" circle, and asynchronous Fetch API for communication.
- **Environment:** We used a portable Python embeddable distribution to ensure the API can run on machines without a global Python installation.

4.4 Challenges and Resolutions

- **VRAM Management:** During testing on an RTX 4070 SUPER, we encountered memory overflow when running both models simultaneously with high batch sizes.
 - *Resolution:* We limited the URL batch size to 1000 and utilized `torch.cuda.empty_cache()` after every email prediction to free up VRAM for the next task.
- **DOM Scrapping Errors:** Gmail frequently changes its internal element IDs.
 - *Resolution:* We implemented a "Page Content Fallback" in `content.js`. If the script fails to find the specific "Body" tag, it extracts the entire `document.documentElement.outerHTML`, allowing the AI (which is trained on noisy data) to parse the relevant text from the HTML tags.

4.5 User Interface Design

The interface was designed to be "Explainable." When a threat is detected, the `displayResults` function dynamically updates the DOM to show:

1. **Color-Coded Score:** 0-40 (Safe/Green), 40-75 (Warning/Yellow), 75+ (Danger/Red).
2. **Keyword Highlighting:** Using a custom `<mark>` class in CSS to highlight suspicious phrases found by the model.
3. **Indicator List:** A breakdown of *why* the email was flagged (e.g., "Urgency Language" or "No-Reply Sender").

Results & Evaluation:

5.1 Testing Strategy

- **Unit Testing:** Verified API health-checks and model loading states.
- **Integration Testing:** Tested the end-to-end flow from Gmail click to threat score display.

5.2 Performance Metrics

Model 1 (DistilBERT - Email Content):	Model 2 (CNN - URL Detection):
● Accuracy: 99.19% ● Precision: 99.19% ● Recall: 99.24% ● F1-Score: 99.21% ● ROC-AUC: 99.94%	● Accuracy: 97.35% ● Precision: 97.5% ● Recall: 97.0% ● F1-Score: 97.5%

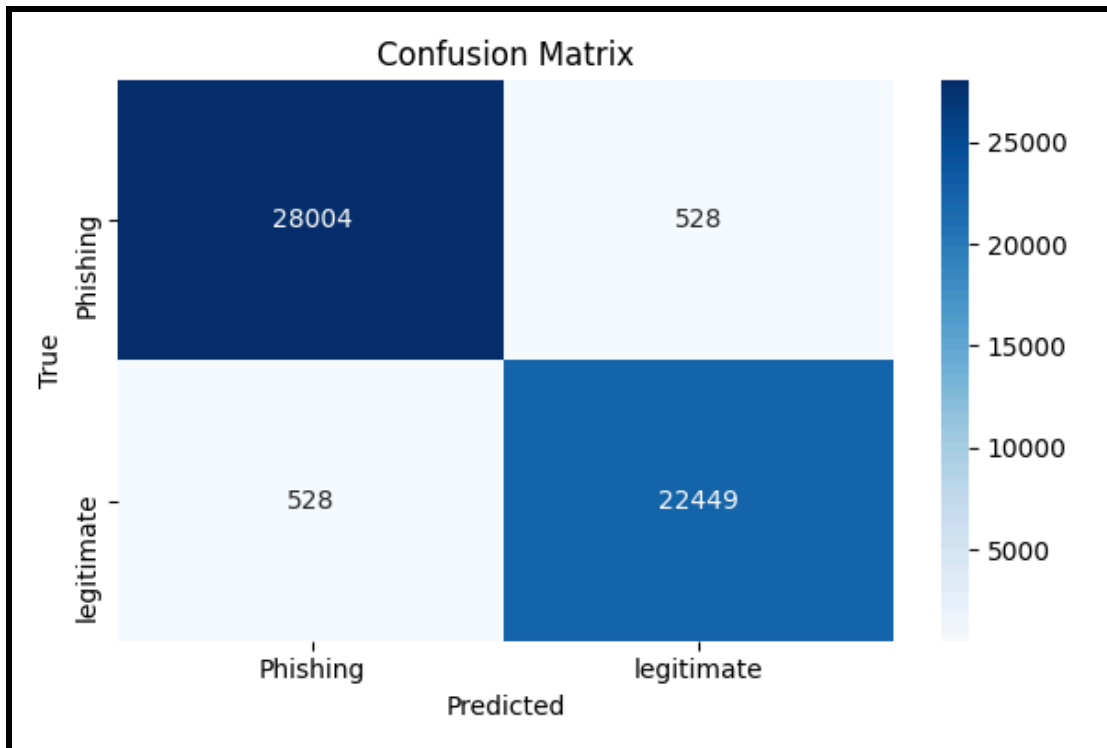


Figure 3: Confusion matrix for model 2 (URL detection).

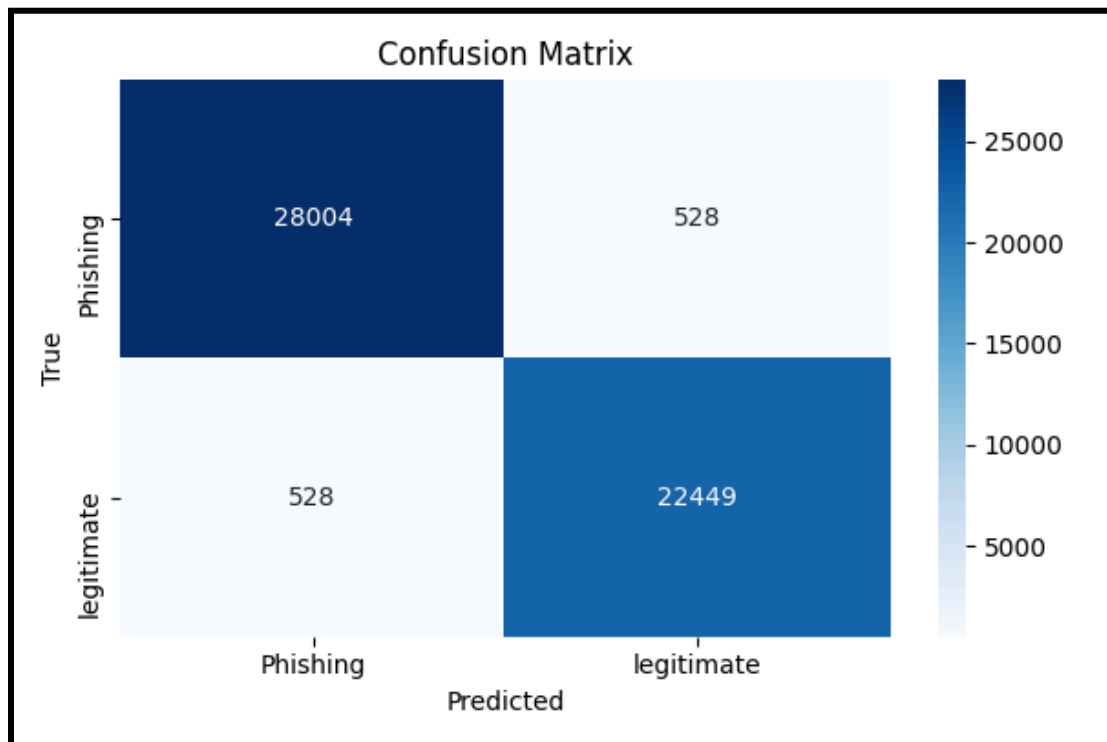


Figure 4: Confusion Matrix for model 1 (email text analysis).

Discussion:

6.1 Objective Reflection

The primary objective of creating a non-intrusive, privacy-first phishing detector was fully met. By processing email content on localhost, PhishGuard achieves a level of privacy that cloud-based competitors cannot provide. The system successfully addresses both semantic risk (language used to trick humans) and structural risk (malicious links).

6.2 Strengths and Weaknesses

- **Strength - Semantic Depth:** The DistilBERT model's ability to recognize urgency and "Authority" impersonation is a significant improvement over traditional heuristic-based filters.
- **Strength - Localized Privacy:** The privacy by design architecture ensures compliance with data protection standards like GDPR.
- **Weakness - Dynamic Content:** The current system relies on static snapshots of the email body. Phishers using highly dynamic JavaScript or Zero-Day obfuscation within an SVG file may occasionally evade the current DOM extraction logic.

6.2 Ethical and Practical Considerations

By performing all analysis on *localhost:5000*, we solved a major ethical dilemma regarding data privacy. Unlike commercial tools, PhishGuard does not require users to upload their private emails to a cloud provider, ensuring General Data Protection Regulation (GDPR) compliance by design.

6.3 Future Work

Future development will focus on Adversarial Training, where the models are exposed to AI-generated phishing lures to improve their detection of synthetic threats. Additionally, we plan to implement a user feedback loop where users can report false negatives to retrain the local model over time.

Conclusion:

7.1 Key Contributions and Findings

PhishGuard successfully demonstrates that a dual-model AI architecture can provide email security within a standard web browser. The primary findings indicate that semantic analysis (DistilBERT) and structural analysis (CNN) are most effective when weighted together, providing a comprehensive defense that neither can achieve in isolation.

7.2 Overall Impact and Significance

The significance of this work lies in its accessibility. By packaging complex transformer models into a Chrome extension with a Flask backend, we have bridged the gap between advanced NLP research and practical cybersecurity. PhishGuard empowers the end-user, turning their browser into an active participant in their defense against digital fraud.

7.3 Final Remarks

As phishing techniques continue to evolve through the use of Large Language Models (LLMs) by attackers, defensive tools must become equally adaptive. PhishGuard provides a modular and scalable foundation for future-proof email security.

References:

- Altwaijry, Najwa, et al. "Advancing Phishing Email Detection: A Comparative Study of Deep Learning Models." *Sensors*, vol. 24, no. 7, Apr. 2024. *Gale Academic OneFile*, dx.doi.org.ccny-proxy1.lib.ccny.cuny.edu/10.3390/s24072077. Accessed 6 Dec. 2025.
- Atawneh, Samer, and Hamzah Aljehani. "Phishing Email Detection Model Using Deep Learning." *Electronics* [Basel], vol. 12, no. 20, Oct. 2023. *Gale Academic OneFile*, dx.doi.org.ccny-proxy1.lib.ccny.cuny.edu/10.3390/electronics12204261. Accessed 6 Dec. 2025.
- Yoon, Jun-Ho, et al. "Phishing Webpage Detection via Multi-Modal Integration of HTML DOM Graphs and URL Features Based on Graph Convolutional and Transformer Networks." *Electronics* [Basel], vol. 13, no. 16, Aug. 2024. *Gale Academic OneFile*, dx.doi.org.ccny-proxy1.lib.ccny.cuny.edu/10.3390/electronics13163344. Accessed 6 Dec. 2025.
- O. K. Sahingoz, E. BUBEr and E. Kugu, "DEPHIDES: Deep Learning Based Phishing Detection System," in *IEEE Access*, vol. 12, pp. 8052-8070, 2024, doi: 10.1109/ACCESS.2024.3352629.
- Alam, Naser Abdullah. "Phishing Email Dataset." Kaggle, 2024.
<https://doi.org/10.34740/KAGGLE/DS/5074342>
- Biber, Ebubekir. "Dephides: Deep Learning-based Phishing Detection System."
GitHub, 2023, <https://github.com/ebubekirbbr/dephides>.